

Trip Plan

Trip Plan est une application web moderne qui permet à un utilisateur de créer un compte, de se connecter et de demander un plan de voyage pour une ville donnée.

Elle combine :

- un backend FastAPI pour la logique métier et l'API
- un frontend Next.js pour l'interface utilisateur
- une intégration avec des services météo et activités

Fonctionnalités

- Inscription d'un utilisateur
- Connexion sécurisée avec token
- Génération d'un plan de voyage pour une ville
- Affichage de la météo et d'activités recommandées
- Interface moderne et responsive

Structure du projet

```
trip_plan/  
├── backend/  
│   ├── app/  
│   │   ├── controllers/  
│   │   ├── core/  
│   │   ├── models/  
│   │   ├── routes/  
│   │   ├── schemas/  
│   │   └── services/  
│   └── main.py  
└── frontend/  
    ├── src/app  
    │   ├── login  
    │   ├── plan  
    │   ├── register  
    └── page.tsx
```

Pages du frontend

- Page d'accueil : présentation du projet et accès aux actions principales
- Page d'inscription : création d'un compte utilisateur
- Page de connexion : authentification utilisateur
- Page de planification : saisie d'une ville et affichage du plan généré

Technologies utilisées

Backend

- Python
- FastAPI
- SQLAlchemy
- Pydantic
- Uvicorn

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Schéma d'architecture



API backend

Le backend expose plusieurs endpoints principaux :

Authentification

- POST /users : créer un utilisateur
- POST /login : connecter un utilisateur et récupérer un token

Planification

- POST /plan : générer un plan de voyage pour une ville donnée, avec authentification Bearer

Prérequis

Avant de lancer le projet, assurez-vous d'avoir installé :

- Python 3.10+
- Node.js 18+
- npm

Configuration

Backend

1. Se placer dans le dossier backend
2. Créer et activer un environnement virtuel
3. Installer les dépendances
4. Configurer les variables d'environnement dans le fichier .env

Exemple :

```
cd backend
python -m venv .env
source .env/bin/activate    # Linux/macOS
.venv\Scripts\activate     # Windows
pip install -r requirements.txt
```

Frontend

```
cd frontend
npm install
```

Lancer le projet

Lancer le backend

```
cd backend
python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Le backend sera disponible sur :

- <http://127.0.0.1:8000>
- Documentation Swagger : <http://127.0.0.1:8000/docs>

Lancer le frontend

```
cd frontend
npm run dev
```

Le frontend sera disponible sur :

- <http://localhost:3000>

Variables d'environnement

Le backend utilise un fichier .env avec des variables telles que :

```
POSTGRES_HOST=localhost
POSTGRES_DB=trip_plan
POSTGRES_USER=postgres
POSTGRES_PASSWORD=your_password
JWT_SECRET=your_secret
CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
```

Exemples d'utilisation

Exemple 1 : créer un compte

```
curl -X POST http://127.0.0.1:8000/users \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}'
```

Exemple 2 : se connecter

```
curl -X POST http://127.0.0.1:8000/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}'
```

Exemple 3 : demander un plan de voyage

```
curl -X POST http://127.0.0.1:8000/plan \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -d '{"city":"Paris"}'
```

Développement

Pour contribuer au projet :

- garder la logique backend propre et testée
- conserver une interface simple et responsive
- vérifier les changements avant de les pousser